Java 入门笔记

第十四课 · 组合、包、对象数组与 static

学习目标　理解组合关系与包机制，会创建对象数组，掌握 static 成员的特点与执行时机。

目录

本课目标 2

一、组合：类的对象作为成员变量 2

二、包（package）机制 5

三、对象数组 6

四、static 关键字 7

五、课上遇到的问题 10

常见错误 10

本课小结 10

自测题 11

整理日期：2026 年 11 月 10 日

> 前几课的成员变量都是 `int`、`String` 这类类型。本课先让一个类的对象成为另一个类的成员（组合），再用包（package）管理越来越多的类，然后把对象放进数组，最后学习属于“类”而不属于某个对象的 `static` 成员。

本课目标

- 理解组合（has-a）关系，会把一个类的对象作为另一个类的成员变量，并正确初始化。
- 理解包的作用，会写 `package` 声明，会用 `import` 或全限定名访问其他包中的类，避开导包的几个误区。
- 理解对象数组存放的是引用，能正确创建并初始化对象数组。
- 掌握静态变量、静态方法和静态初始化器的特点与执行时机。

一、组合：类的对象作为成员变量

1. 概念

成员变量的类型不只能是 `int`、`double`、`String`，也可以是自己写的类。类本身就是一种自定义的类型：既然 `String name;` 可以作为成员变量，`Car car;` 当然也可以。

一个类中包含另一个类的对象作为成员，称为组合，表示 **has-a**（有一个） 关系：学生有一辆车、有一部手机。包含成员对象的类称为组合类，被包含的对象称为成员对象。

> 组合与以后要学的继承不同：继承表示 is-a（是一个），例如“学生是一个人”；组合表示 has-a，例如“学生有一部手机”。

2. 课堂代码

三个类都放在 `class14.demo` 包中，每个类一个文件。

Car.java

```
package class14.demo;

public class Car {
    private String color;
    private int speed;

    public Car(String color, int speed) {
        this.color = color;
        this.speed = speed;
    }

    public void run() {
        System.out.println(this.color + "的车，时速" + this.speed);
    }
}
```

Phone.java

```
package class14.demo;

public class Phone {
    private String number;
    private String brand;

    public Phone(String number, String brand) {
        this.number = number;
        this.brand = brand;
    }

    public void call() {
        System.out.println("用" + brand + "手机，号码" + number + "打电话");
    }
}
```

Student.java

```
package class14.demo;

// 一个类的对象可以作为另一个类的成员变量（组合，has-a）
public class Student {
    private String name;
    private int age;
    private double grade;
    private Car car;          // Car 类的对象作为成员变量
    private Phone phone;      // Phone 类的对象作为成员变量

    public Student(String name, int age, double grade, Car car, Phone phone) {
        this.name = name;
        this.age = age;
        this.grade = grade;
        this.car = car;
        this.phone = phone;
    }

    public void display() {
        System.out.println("姓名：" + name + "，年龄：" + age + "，成绩：" + grade);
        car.run();            // 通过成员变量 car 调用 Car 的方法
        phone.call();         // 通过成员变量 phone 调用 Phone 的方法
    }
}
```

class14.java

```
package class14.demo;

public class class14 {
    public static void main(String[] args) {
        Car c1 = new Car("红色", 180);
        Phone p1 = new Phone("13800000000", "华为");
        Student s = new Student("张三", 21, 74, c1, p1);
        s.display();
    }
}
```

运行结果

```
姓名：张三，年龄：21，成绩：74.0
红色的车，时速180
用华为手机，号码13800000000打电话
```

执行过程：

1. 先 `new` 出车 `c1` 和手机 `p1`。
2. 创建学生时把 `c1`、`p1` 作为实参传入构造方法，保存到成员变量 `car`、`phone` 中。
3. `display()` 中通过 `car.run()`、`phone.call()` 使用这两个成员对象。

几点说明：

- 成员变量 `car` 中保存的是 Car 对象的地址（引用），并没有把整个对象复制一份放进学生对象。所以 `s` 中的 `car` 与 `main` 中的 `c1` 指向同一辆车。
- 成绩显示为 `74.0`，因为 `grade` 是 `double`，整数实参 74 自动转换成了小数。
- 访问链从左往右读：`s.display()` → 学生的 `car` → 车的 `run()`。

3. 组合类成员对象的初始化

成员对象是引用类型，没有初始化时默认值为 `null`。对 `null` 调用方法会抛出 **NullPointerException**（空指针异常）。所以成员对象在使用前必须指向一个真正的对象。常用两种方式：

① 构造方法初始化：创建组合类对象时由外部传入，最常用，课堂代码就是这种方式。

```
public Student(String name, Phone phone) {
    this.name = name;
    this.phone = phone;       // 由调用者决定是哪部手机
}
```

② 声明初始化：声明成员变量时直接 `new` 出对象，适合每个对象都需要一个默认成员对象的情况。

```
public class Student {
    private Car car = new Car("白色", 120);   // 每个学生默认都有一辆白色的车
    ...
}
```

方式	写法	特点
构造方法初始化	构造方法中 `this.car = car;`	由外部传入，每个对象可以不同，更灵活
声明初始化	`private Car car = new Car(...);`	写法简单，但每个对象得到的初值都一样

> 课上加 phone 时的典型错误：只写了 `private Phone phone;`，构造方法中没有 `this.phone = phone;`。代码能编译，但 phone 一直是 null，一调用 `phone.call()` 就抛空指针异常。

二、包（package）机制

1. 为什么需要包

- 解决命名冲突：不同的人、不同的模块都可能写出叫 `Car`、`Student` 的类。只要它们在不同的包中，就可以同时存在，例如 `class13.Car` 与 `class14.demo.Car`。
- 分类管理：把功能相关的类放进同一个包，项目结构更清晰。
- 控制访问：默认权限和 `protected` 都以包为边界（见第 13 课访问权限表）。

2. 声明包

```
package 包名;
```

- 必须是源文件中的第一条语句（注释除外）。
- 一个源文件只能有一条 `package` 语句。
- 包名对应文件夹：`package class14.demo;` 的源文件必须放在 `class14/demo/` 目录中。
- 命名习惯：全部小写，多级之间用 `.` 分隔；公司项目常用倒写的域名开头，如 `com.example.project`。
- 类的完整名字是“包名.类名”，称为全限定名，例如 `class14.demo.Car`。

3. 不同包之间的互相访问

同一个包中的类可以直接使用，不需要导入。不同包中的类要访问，需要满足：

1. 被访问的类必须是 `public`。没有修饰符的类只能在本包内使用。
2. 使用时导包，或者写全限定名。

方式一：导包（**import**）

```
package class14.test;

import class14.demo.Car;        // 导入单个类（推荐）
// import class14.demo.*;       // 导入 demo 包中的所有类

public class Main {
    public static void main(String[] args) {
        Car c = new Car("蓝色", 120);
        c.run();
    }
}
```

`import` 语句写在 `package` 之后、类定义之前。

方式二：全限定名

```
class14.demo.Car c = new class14.demo.Car("蓝色", 120);
```

不需要 `import`，但每次都要写完整的包名，一般只在发生命名冲突时使用。

导入了类，不代表它的所有成员都能用，成员还要看访问修饰符：从其他包访问 `Car` 时，`public` 的构造方法和 `run()` 可以使用；`private` 的 `color` 无法访问；去掉 `public` 的方法也无法访问。

4. 导包的误区

误区一：星号导入包含子包

`*` 只导入该包本身的类，不包括子包中的类。

```
import class14.*;          // 只导入 class14 包下的类
                           // class14.demo 中的 Car、Student 并没有被导入
import class14.demo.*;     // 要用 demo 中的类，必须单独导入子包
```

包的层级只是名字上的层级，`class14` 和 `class14.demo` 在 Java 看来是两个独立的包。

误区二：命名冲突

两个不同包中有同名的类，同时导入会产生冲突。最典型的是 `java.util.Date` 和 `java.sql.Date`：

```
import java.util.Date;
import java.sql.Date;      // ✗ 编译报错：Date 已经从 java.util 导入
```

解决：只导入其中一个，另一个用全限定名。

```
import java.util.Date;

Date d1 = new Date();                    // java.util.Date
java.sql.Date d2 = new java.sql.Date(0);  // java.sql.Date 用全限定名
```

误区三：冗余导入

以下导入都是多余的，删掉也能正常编译：

- `java.lang` 包中的类，如 `String`、`System`、`Math`，编译器会自动导入。
- 同一个包中的类，可以直接使用。
- 同一个类导入了两次。

IDEA 中灰色显示的 `import` 就是没有用到的冗余导入，可以用 `Ctrl + Alt + O`（macOS 为 `Ctrl + Option + O`）一键清理。

三、对象数组

1. 概念

元素类型是类的数组，称为对象数组。对象数组的元素是对象的引用：数组中不存放对象本身，只存放指向对象的“地址”。

2. 定义与初始化

对象数组要分两步完成：先创建数组，再逐个创建对象。

```
// 第一步：声明并创建数组空间（此时 3 个元素全为 null，还没有任何 Car 对象）
Car[] cars = new Car[3];
System.out.println(cars[0]);    // null

// 第二步：逐个初始化数组元素（创建真正的对象）
for (int i = 0; i < cars.length; i++) {
    cars[i] = new Car("颜色" + i, 100 + i * 20);
}

// 遍历使用
for (Car c : cars) {
    c.run();
}
```

运行结果

```
null
颜色0的车，时速100
颜色1的车，时速120
颜色2的车，时速140
```

`new Car[3]` 只创建了能放 3 个引用的“格子”，并没有创建 **3** 辆车。只完成第一步就调用 `cars[0].run()`，会抛出空指针异常。

也可以在声明时直接初始化：

```
Car[] cars2 = { new Car("红色", 180), new Car("蓝色", 120) };
```

> 老师板书写的是 `cars[i] = new Car();`，这要求 `Car` 有无参构造方法。本课的 `Car` 只写了带两个参数的构造方法，编译器不再提供无参构造（第 13 课的“默认构造方法”陷阱），所以这里要传参，或者给 `Car` 补一个无参构造。

3. 内存示意

位置	存放的内容
栈	变量 `cars`：保存数组对象的地址
堆：数组对象	3 个元素 `cars[0]`～`cars[2]`，每个元素只保存一个 Car 对象的地址（未初始化时为 `null`）
堆：Car 对象	3 个各自独立的 Car 对象，各有自己的 `color` 和 `speed`

访问 `cars[1].run()` 的过程：通过 `cars` 找到数组 → 取出 `cars[1]` 中的地址 → 找到对应的 Car 对象 → 调用它的 `run()`。

四、static 关键字

1. 修饰成员变量：静态变量

用 `static` 修饰的成员变量称为静态变量，也叫类变量。

- 属于类，不属于某个对象：无论创建多少个对象，静态变量在内存中只有一份，所有对象共享。
- 共享存储，高效利用：所有对象都相同的数据（如学校名称、对象计数）没必要在每个对象中各存一份。

- 存储在方法区，随类的加载而创建，不依赖对象。普通成员变量（实例变量）存储在堆中的每个对象里。
- 推荐用类名访问：`类名.静态变量`。用对象名也能访问，但容易误以为它属于对象。

对比	实例变量（无 **static**）	静态变量（有 **static**）
属于	每个对象	类
份数	每个对象各一份	整个类只有一份
存储位置	堆（对象内部）	方法区
创建时机	`new` 对象时	类加载时
访问方式	`对象名.变量`	`类名.变量`（推荐）

2. 修饰方法：静态方法

用 `static` 修饰的方法称为静态方法，也叫类方法，用 `类名.方法名()` 调用，不需要创建对象。`main` 方法就是静态方法。

静态方法不能直接访问非静态成员（实例变量和实例方法），也不能使用 `this`。原因：静态方法属于类，调用时可能还没有任何对象，自然不知道该访问哪个对象的成员。

```
static void showCount() {
    System.out.println(count);    // ✓ 静态方法访问静态变量
    // System.out.println(name);  // ✗ 编译报错：无法从静态上下文中引用非静态变量
}
```

反过来，实例方法可以访问静态成员，因为对象存在时，类一定已经加载了。

调用方	访问静态成员	访问实例成员
静态方法	✓ 直接访问	✗ 必须先创建对象，再用 `对象.成员`
实例方法	✓ 直接访问	✓ 直接访问

3. 静态初始化器（静态代码块）

用 `static { ... }` 定义的代码块称为静态初始化器。

- 执行时机：类加载时自动执行，也就是这个类第一次被使用的时候，早于构造方法。
- 执行次数：只执行 **1** 次，无论创建多少个对象。
- 用途：初始化静态变量、加载驱动、读取配置文件等只需要做一次的准备工作。

4. 综合示例

```
class Counter {
    String name;                    // 实例变量：每个对象一份
    static int count = 0;           // 静态变量：所有对象共享
    static String school;

    static {                        // 静态初始化器：只执行一次
        school = "XX 学院";
        System.out.println("静态初始化器执行");
    }

    Counter(String name) {
        this.name = name;
        count++;                    // 每创建一个对象，共享的计数加 1
        System.out.println("构造方法执行：" + name);
    }

    static void showCount() {       // 静态方法
        System.out.println("共创建 " + count + " 个对象");
    }
}

public class Demo {
    public static void main(String[] args) {
        System.out.println("main 开始");
        Counter a = new Counter("A");
        Counter b = new Counter("B");
        Counter.showCount();        // 用类名调用静态方法
        a.count = 10;               // 通过 a 修改静态变量
        System.out.println(b.count);
    }
}
```

运行结果

```
main 开始
静态初始化器执行
构造方法执行：A
构造方法执行：B
共创建 2 个对象
10
```

从结果可以看出：

- “静态初始化器执行”出现在“main 开始”之后、第一个构造方法之前，因为 `Counter` 类是第一次 `new` 它时才加载的；而且只输出了一次。
- 两个对象共享同一个 `count`，所以计数为 2。
- 通过 `a` 把 `count` 改成 10，`b.count` 也变成了 10，证明只有一份。

五、课上遇到的问题

现象	原因	解决
`new Student("张三"，21，74，c1)` 报错	① 构造方法只有 3 个形参，却传了 4 个实参；② 用了中文逗号	给构造方法增加 `Car car` 形参；改用英文逗号
加了 `private Phone phone;`，手机却没有用上	只声明，没有在构造方法中赋值，也没有在 `display()` 中调用	构造方法增加形参并 `this.phone = phone;`，`display()` 中调用 `phone.call()`
构造方法改成 5 个参数后，`main` 报错	`main` 中仍然只传 4 个实参	先 `new` 一部手机，再作为第 5 个实参传入
`Phone c1 = ...` 下面红线，`p1` 也标红	同一个方法中已经有变量 `c1`，不能重复声明；`p1` 从未声明	把手机变量命名为 `p1`
一个文件里写多个类时，`Car`、`Student` 不能加 `public`	一个源文件只能有一个 `public` 类，且必须与文件名相同	去掉 `public`，或者每个类单独一个文件

常见错误

- 成员对象只声明不初始化，调用其方法时抛出空指针异常。
- 构造方法的参数个数、类型、顺序与调用时的实参不一致。
- 代码中混入中文标点（逗号、分号、括号、引号）。
- 以为 `import 包名.*;` 会连同子包一起导入。
- 同时导入两个包中的同名类，例如 `java.util.Date` 与 `java.sql.Date`。
- 从其他包访问没有 `public` 修饰的类或方法。
- 以为 `new Car[3]` 已经创建了 3 个 Car 对象。
- 在静态方法中直接访问实例变量，或者使用 `this`。
- 以为静态初始化器每次 `new` 对象都会执行。

本课小结

- 组合（has-a）：一个类的对象作为另一个类的成员变量，保存的是引用；成员对象可以通过构造方法或声明时初始化，使用前不能是 `null`。
- 包：`package` 是源文件第一条语句，包名对应文件夹；不同包之间访问需要 `public`，再用 `import` 或全限定名；星号导入不含子包。
- 对象数组存的是引用：先 `new` 数组，再逐个 `new` 对象。
- `static` 成员属于类，只有一份，用类名访问；静态方法不能直接访问非静态成员；静态初始化器在类加载时执行，只执行一次。

自测题

1. 组合与继承分别表示什么关系？各举一个例子。
2. 成员变量 `private Phone phone;` 没有初始化，调用 `phone.call()` 会怎样？列出两种初始化方式。
3. 写了 `import class14.*;`，能否直接使用 `class14.demo` 中的 `Car`？为什么？
4. 同一个类中要同时使用 `java.util.Date` 和 `java.sql.Date`，应该怎样写？
5. 执行 `Car[] cars = new Car[5];` 后，内存中有几个 Car 对象？`cars[0]` 的值是什么？
6. 某类有 `static int count;`，创建了 3 个对象，内存中有几个 `count`？
7. 静态方法中为什么不能使用 `this`？
8. 一个类有静态初始化器，程序中创建了 10 个该类的对象，静态初始化器执行几次？在什么时候执行？